목차

머릿말 1

프롤로그 2

챕터 1. MSA란 무엇인가? 7

1.1 모놀리식 - 쇼핑몰을 하나의 서버로 만들면 어떻게 될까? 10

1.1.1 처음에는 아무 문제가 없었다 10

1.1.2 성장하면서 균열이 생긴다 10

1.2 마이크로서비스 - 역할을 나눈다 11

1.2.1 백화점 vs 개별 상점 11

1.3 시스템과 핵심 과제 13

1.3.1 쇼핑몰 주문 시스템 13

1.3.2 서비스 간 요청의 두 가지 방식 13

1.3.3 분산 트랜잭션 — 이 책의 핵심 과제 15

1.4 이 책의 학습 흐름 16

챕터 2. 동기식 MSA 구현 18

2.1 공통 설정 - 모든 서비스가 공유하는 뼈대 21

2.2 회원 서비스 - JWT로 로그인하다 22

2.2.1 클래스 구성 22

2.3 상품 서비스 - 재고를 관리하다 23

2.3.1 클래스 구성 23

2.4 배달 서비스 - 배달을 생성하고 취소하다 23

2.4.1 클래스 구성 24

2.5 주문 서비스 - 보상 트랜잭션의 현장 24

2.5.1 클래스 구성 25

2.5.2 보상 트랜잭션 흐름 25

2.5.3 adapter - 외부 서비스 호출 설정 26

2.5.4 OrderService - 보상 트랜잭션의 핵심 27

2.6 Docker Compose - 네 개의 서비스를 한 번에 실행하다 28

2.6.1 서비스 실행 29

2.6.2 Hoppscotch와 인터셉터 설정 30

2.6.3 시나리오 1: 정상 주문 31

2.6.4 시나리오 2: 재고 부족 35

2.6.5 시나리오 3: 주소 누락 35

챕터 3. 도메인을 중심으로 38

3.1 도메인 주도 개발(Domain-Driven Design) - 비즈니스 로직을 도메인으로 41

3.2 UseCase 인터페이스(Use Case Interface) - USB 허브 42

3.3 패키지 구조 비교 - 단일 패키지에서 책임별 패키지로 44

3.4 UseCase 인터페이스 + 도메인 캡슐화 도입 45

3.4.1 UseCase 인터페이스 정의 45

3.4.2 엔티티의 비즈니스 로직 — DDD의 핵심 45

3.4.3 OrderService - 인터페이스 구현 46

3.4.4 OrderController 수정 46

3.5 Gateway와 MySQL 인프라 47

3.5.1 Nginx - API Gateway 라우팅 47

3.5.2 MySQL - 데이터베이스 인프라 48

3.6 Kubernetes - YAML로 선언하는 배포 49
3.6.1 리소스 구조 설계 49
3.7 Minikube - 실행 및 결과 확인 51
3.7.1 Minikube 시작 51
3.7.2 이미지 빌드 51
3.7.3 배포 순서 52
3.7.4 배포 상태 확인 53
3.7.5 서비스 접근 54

챕터 4. 비동기 MSA 57
4.1 동기 호출의 한계 - 한 서비스가 멈추면 전체가 멈춘다 60
4.1.1 카운터 대기 vs 진동벨 61
4.2 Kafka - 메시지를 전달하는 우체국 61
4.2.1 프로듀서·토픽·컨슈머 61
4.2.2 컨슈머 그룹 63
4.3 Orchestration Saga - 지휘자가 흐름을 조율하다 65
4.3.1 이번 챕터에서 사용하는 토픽 맵 66
4.3.2 주문 요청 성공 흐름 66
4.3.3 주문 요청 실패 흐름 (보상 트랜잭션) 69
4.4 Kafka로 주고받기 - 발행과 구독 70
4.4.1 발행 - 토픽에 메시지를 넣는다 70
4.4.2 orchestrator - 흐름을 조율하는 코드 72
4.4.3 구독 - 토픽의 메시지를 받는다 72
4.5 Kubernetes - Kafka와 orchestrator 배포 73
4.6 실행 및 결과 확인 74
4.6.1 이미지 빌드 74
4.6.2 배포 순서 75
4.6.3 서비스 접근 76
4.6.4 비동기 흐름 테스트 77
4.6.5 보상 트랜잭션 확인 - 품절 상품 주문 78
4.7 더 알아보기 - Outbox 패턴 80

챕터 5. 실시간 알림 83
5.1 웹소켓 - 폴링의 한계를 넘다 86
5.1.1 폴링 vs 푸시 86
5.1.2 웹소켓 - 실시간 양방향 통신 87
5.2 배달 완료 - 생성과 완료를 분리한다 88
5.3 웹소켓 연결 흐름 89
5.3.1 브라우저와 주문 서비스의 연결 - HTTP에서 웹소켓으로 89
5.3.2 브라우저의 채널 구독 - 명부에 등록 90
5.3.3 주문 서비스의 알림 발송 - 같은 채널 찾아 전달 90
5.4 배달 서비스 - 배달 완료 API 91
5.4.1 createDelivery 수정 - 배달 생성·완료 분리 91
5.4.2 completeDelivery 추가 - 배달 완료 API 92
5.5 orchestrator - 배달 완료 이벤트 처리 추가 92
5.6 주문 서비스 - STOMP로 실시간 Push 구현 93
5.6.1 웹소켓 설정 93
5.6.2 주문 완료 시 알림 발송 94
5.7 프론트엔드 연결 94
5.7.1 업그레이드 헤더 전달 94
5.7.2 클라이언트 - STOMP 구독 95
5.8 전체 시스템 통합 테스트 96

5.8.1 Kubernetes 리소스 정의 96
5.8.2 이미지 빌드 97
5.8.3 배포 97
5.8.4 서비스 접근 98
5.8.5 통합 테스트 시나리오 98

에필로그 107

마치며 108